Demo
Bidirectional Encoder Representations from Transformers (BERT)

Sentiment analysis
IMDB reviews

IMDB reviews dataset

imdb_reviews dataset is a large movie review dataset. This dataset is for binary sentiment classification containing:

- Training set: 25,000 movie reviews.
- Test set: 25,000 movie reviews.

All the reviews have either a positive (1) or negative (0) sentiment.

Reviews	Label
This was an absolutely **terrible movie**. Don't be lured in by Christopher Walken or Michael Ironside. Both are great actors, but this must simply be their **worst role** in history. Even their great acting could not redeem this movie's **ridiculous storyline**. This movie is an early nineties US propaganda piece. The most pathetic scenes were those when the Columbian rebels were making their cases for revolutions. Maria Conchita Alonso appeared phony, and her pseudo-love affair with Walken was nothing but a pathetic emotional plug in a movie that was devoid of any real meaning. I am disappointed that there are movies like this, ruining actor's like Christopher Walken's good name. I could barely sit through it.	0
I have been known to **fall asleep** during films, but this is usually due to a combination of things including, really tired, being warm and comfortable on the sette and having just eaten a lot. However on this occasion I **fell asleep** because the film was **rubbish**. The plot development was constant. Constantly **slow and boring**. Things seemed to happen, but with no explanation of what was causing them or why. I admit, I may have missed part of the film, but i watched the majority of it and everything just seemed to happen of its own accord without any real concern for anything else. I **cant recommend** this film at all.	0
This is the kind of film for a snowy Sunday afternoon when the rest of the world can go ahead with its own business as you descend into a big arm-chair and mellow for a couple of hours. **Wonderful performances** from Cher and Nicolas Cage (as always) gently row the plot along. There are no rapids to cross, no dangerous waters, just a warm and witty paddle through New York life at its best. A family film in every sense and one that **deserves the praise** it received.	1
As others have mentioned, all the women that go nude in this film are mostly absolutely **gorgeous**. The plot very ably shows the hypocrisy of the female libido. When men are around they want to be pursued, but when no "men" are around, they become the pursuers of a 14 year old boy. And the boy becomes a man really fast (we should all be so lucky at this age!). He then gets up the courage to pursue his true love.	1

Model training

Target: build a classifier to detect whether a review has positive or negative sentiment.

Training

IMDB reviews Training set

process data

Classifier

"The movie was great"

A new movie review

process data

Trained Classifier

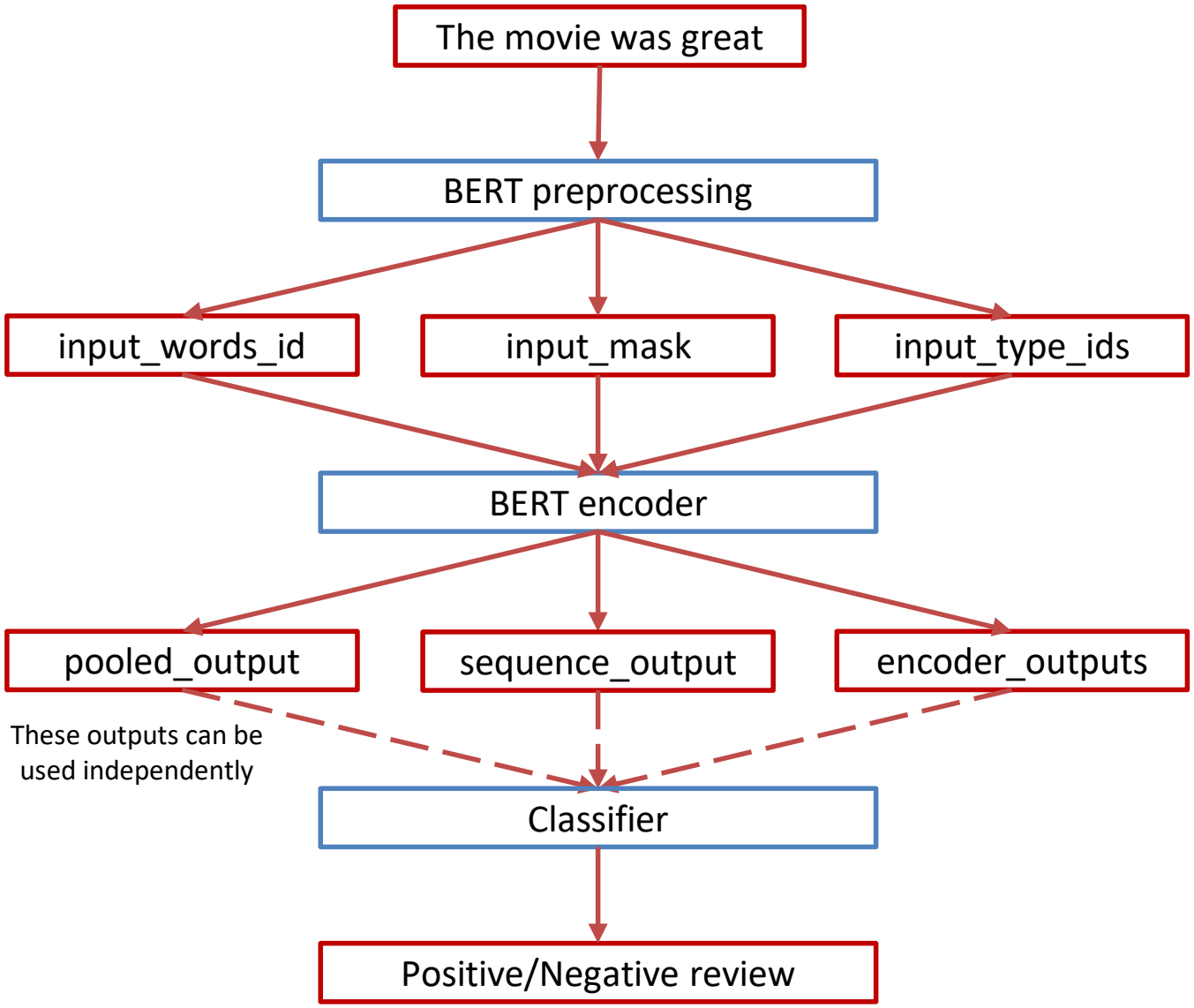
predict

Positive/ Negative

Prediction

BERT demo code

Data processing flow



Input text movie review

Preprocess text input using BERT preprocessing module

Output of BERT preprocessing module

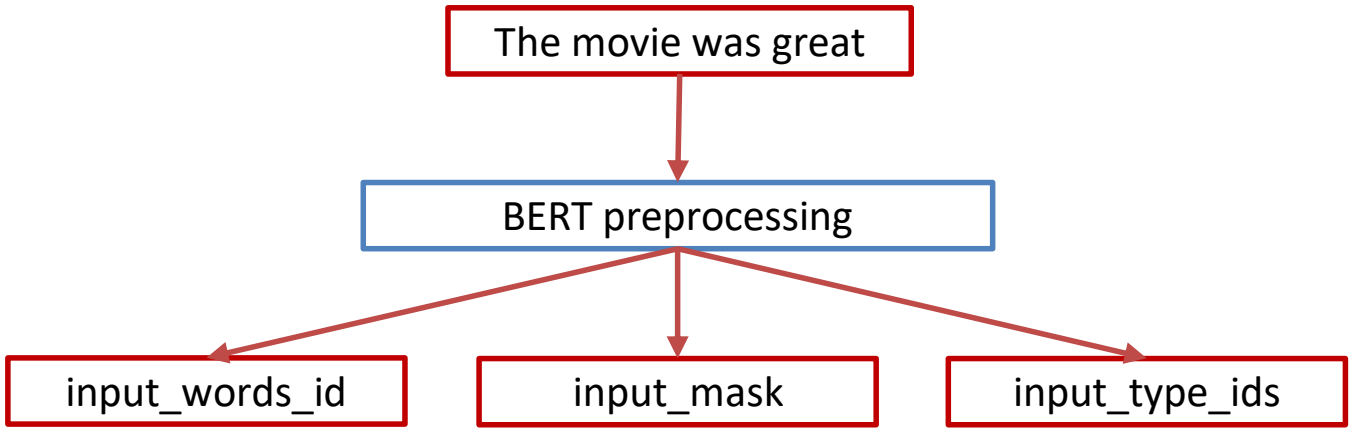
Produce embedding vectors using BERT encoder

Output of BERT encoding module

Classify text movie review using embedding vectors

Output sentiment label

BERT processing - input_word_ids



Input text movie review

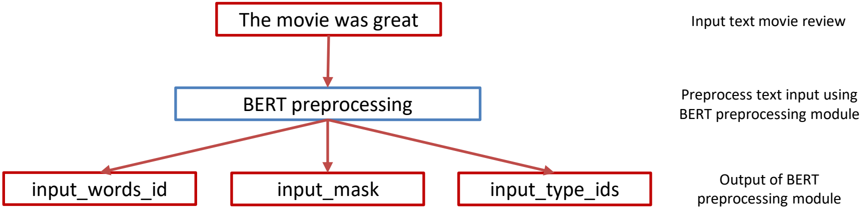
Preprocess text input using BERT preprocessing module

Output of BERT preprocessing module

input_word_ids: give unique numerical ids for each tokenized input, including start [CLS] of 101, end [SEP] of 102 and padding tokens. Its shape is (number of sentences, 128) where 128 is the maximum length and configurable.

```
input_word_ids: tf.Tensor(
[[ 101 2023 3185 2001 2307  102    0    0    0    0    0    0    0    0
     0    0    0    0    0    0    0    0    0    0    0    0    0    0
     0    0    0    0    0    0    0    0    0    0    0    0    0    0
     0    0    0    0    0    0    0    0    0    0    0    0    0    0
     0    0    0    0    0    0    0    0    0    0    0    0    0    0
     0    0    0    0    0    0    0    0    0    0    0    0    0    0
     0    0    0    0    0    0    0    0    0    0    0    0    0    0
     0    0    0    0    0    0    0    0    0    0    0    0    0    0
     0    0    0    0    0    0    0    0    0    0    0    0    0    0
     0    0]], shape=(1, 128), dtype=int32)
```

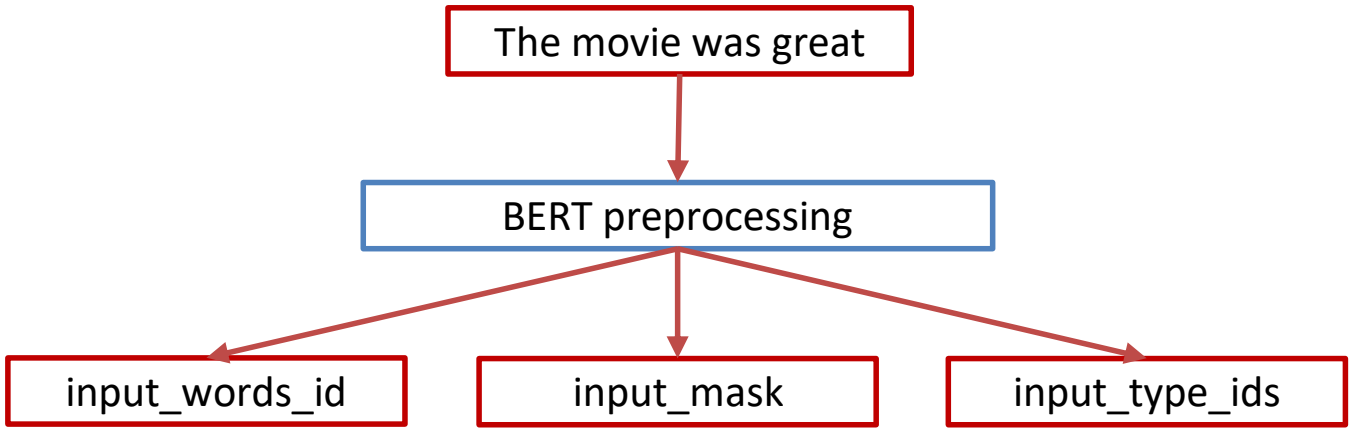
BERT processing - input_mask



input_mask: tells non-padding from padding tokens. Its shape is (number of sentences, 128) where 128 is the maximum length and configurable.

```
input_mask: tf.Tensor(
[[1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
  0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
  0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
  0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]], shape=(1, 128), dtype=int32)
```

BERT processing - input_type_ids



Input text movie review

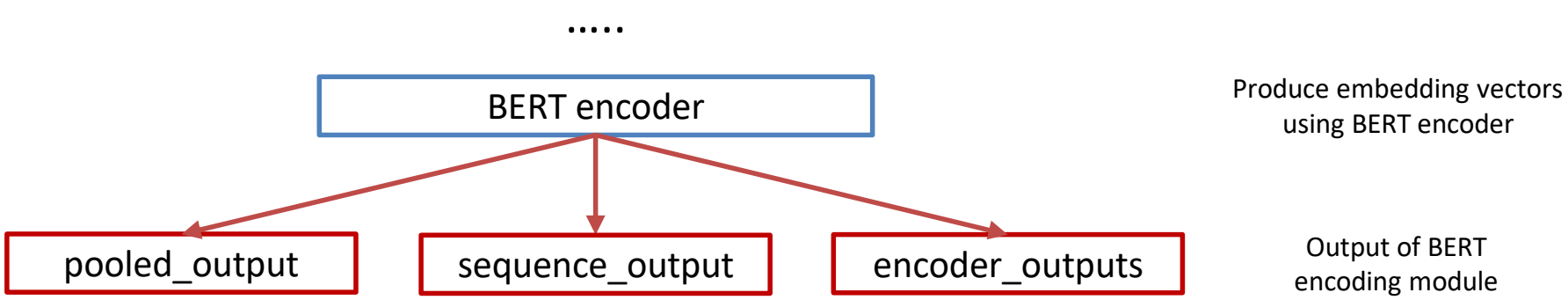
Preprocess text input using BERT preprocessing module

Output of BERT preprocessing module

input_type_ids: to combine multiple sentences into one data sample for a more efficient training process. In this case, we only have one value '0' because this is a single sentence input. For a multiple-sentence input, it would have one number for each input. Its shape is (number of sentences, 128) where 128 is the maximum length and configurable.

```
input_type_ids: tf.Tensor(
[[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
  0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
  0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
  0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]], shape=(1, 128), dtype=int32)
```

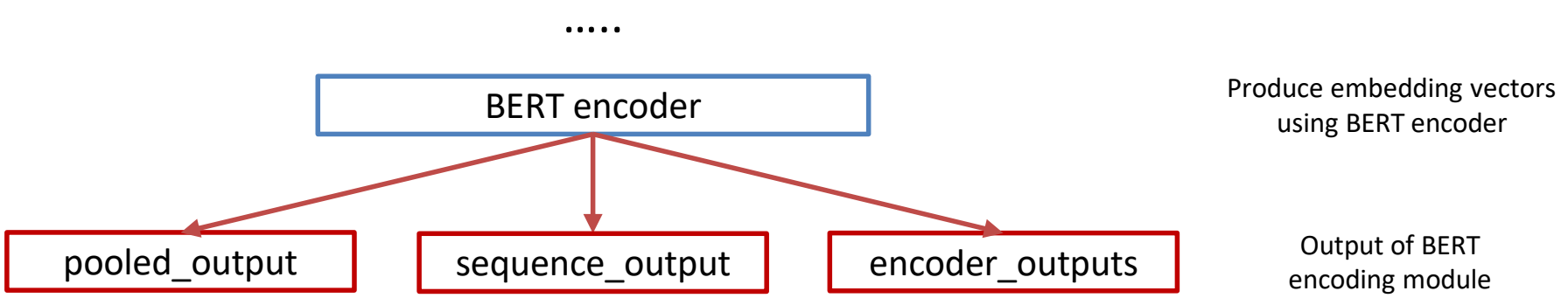
BERT encoder - pooled_output



pooled_output: represents each input sequence as a whole. The shape is [batch_size, hidden_state]. This can be seen as an embedding for the entire movie review.

```
pooled_output shape: (1, 128)
pooled_output values: tf.Tensor(
[[-9.9993491e-01  7.8811862e-02 -9.9978060e-01  9.9903172e-01
  -9.9980986e-01  9.7693634e-01 -9.9760538e-01 -1.4675389e-01
   1.2690674e-01 -7.2680414e-03 -7.9470134e-01 -1.4446791e-02
   1.0650654e-02  9.9999964e-01 -8.5033560e-01 -9.9351531e-01
   9.3729025e-01  4.6182487e-02 -9.8182863e-01  9.5978212e-01
   9.7017992e-01  6.0048834e-03  9.9541026e-01  2.2610748e-01
  -9.9979001e-01 -1.2126343e-02 -9.9924058e-01  8.2546604e-01
```

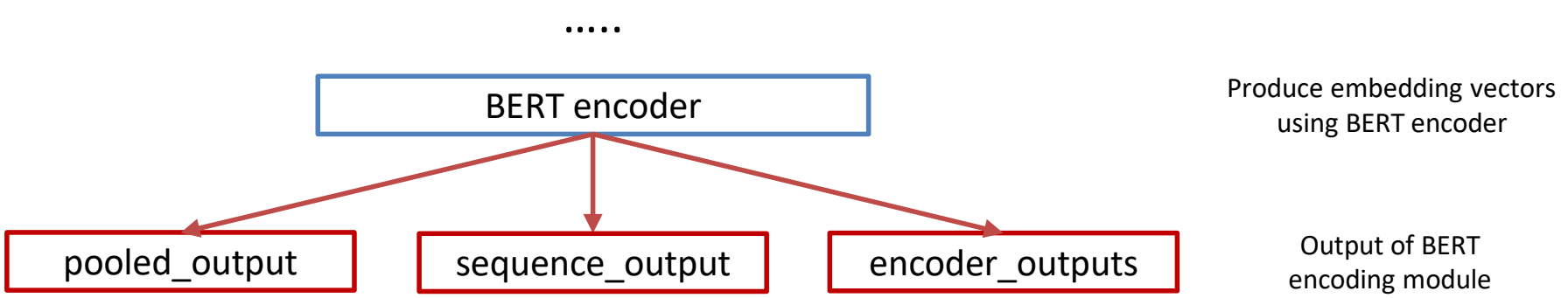
BERT encoder - sequence_output



sequence_output: represents each input token in the context. The shape is [batch_size, seq_length, hidden_states]. This can be seen as a contextual embedding for every token (word) in the movie review.

```
sequence_output shape: (1, 128, 128)
sequence_output values: tf.Tensor(
[[[-0.5366354  -1.0478935  -3.1254625  ... -0.90370464 -2.1683738
    1.0213721 ]
  [-1.4954466  -0.26101232  0.35833746 ... -2.2641153  -2.2700477
   -0.05983699]
  [-1.3746109  -1.0767092   0.20740092 ... -1.8749114  -2.11168
    0.97796434]
  ...
  [ 0.01303321 -0.87776643 -0.3316344  ... -1.59137    -0.7492585
    2.0238166 ]
  [ 0.12492017 -0.4953426  -0.2531013  ... -1.277288   -0.8560324
    1.969287  ]
  [-0.1092886  -0.2830223  -0.45269978 ... -1.0589341  -1.4218843
    2.092354  ]]], shape=(1, 128, 128), dtype=float32)
```

BERT encoder - sequence_output



sequence_output: represents each input token in the context. The shape is [batch_size, seq_length, hidden_states]. This can be seen as a contextual embedding for every token (word) in the movie review.

```
sequence_output shape: (1, 128, 128)
sequence_output values: tf.Tensor(
[[[-0.5366354  -1.0478935  -3.1254625  ... -0.90370464 -2.1683738
    1.0213721 ]
  [-1.4954466  -0.26101232  0.35833746 ... -2.2641153  -2.2700477
   -0.05983699]
  [-1.3746109  -1.0767092   0.20740092 ... -1.8749114  -2.11168
    0.97796434]
  ...
  [ 0.01303321 -0.87776643 -0.3316344  ... -1.59137    -0.7492585
    2.0238166 ]
  [ 0.12492017 -0.4953426  -0.2531013  ... -1.277288   -0.8560324
    1.969287  ]
  [-0.1092886  -0.2830223  -0.45269978 ... -1.0589341  -1.4218843
    2.092354  ]]], shape=(1, 128, 128), dtype=float32)
```

Model summary

```
Model: "BERT_complete_model"
__________________________________________________________________________________________________
 Layer (type)                   Output Shape         Param #     Connected to
==================================================================================================
 text (InputLayer)              [(None,)]            0           []

 preprocessing (KerasLayer)     {'input_type_ids':   0           ['text[0][0]']
                                (None, 128),
                                 'input_mask': (Non
                                e, 128),
                                 'input_word_ids':
                                (None, 128)}

 BERT_encoder (KerasLayer)      {'pooled_output': (  4385921     ['preprocessing[0][0]',
                                None, 128),                      'preprocessing[0][1]',
                                 'sequence_output':              'preprocessing[0][2]']
                                 (None, 128, 128),
                                 'encoder_outputs':
                                 [(None, 128, 128),
                                 (None, 128, 128)],
                                 'default': (None,
                                128)}

 dense_7 (Dense)                (None, 64)           8256        ['BERT_encoder[0][3]']

 batch_normalization_1 (BatchNo  (None, 64)          256         ['dense_7[0][0]']
 rmalization)

 dropout_7 (Dropout)            (None, 64)           0           ['batch_normalization_1[0][0]']

 classifier (Dense)             (None, 1)            65          ['dropout_7[0][0]']

==================================================================================================
Total params: 4,394,498
Trainable params: 4,394,369
Non-trainable params: 129
__________________________________________________________________________________________________
```

small_bert/bert_en_uncased_L-2_H-128_A-2

Use **pooled_output** embedding as input of classification layers

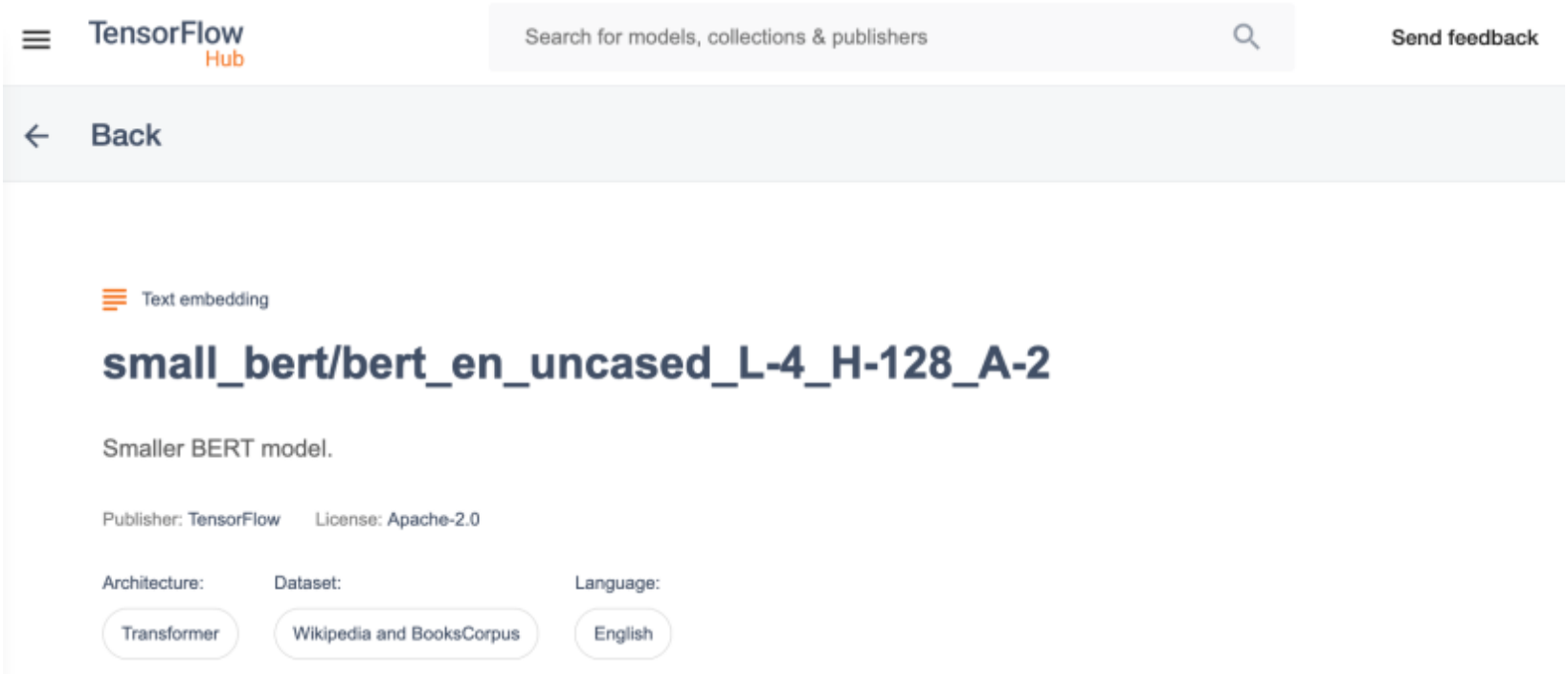
BERT

Assignment instruction

Assignment description

In this assignment, you will exploit **multi-layer feature extraction** of a BERT model which comprises of several Transformer-encoder blocks. The model you will be using is as follows:

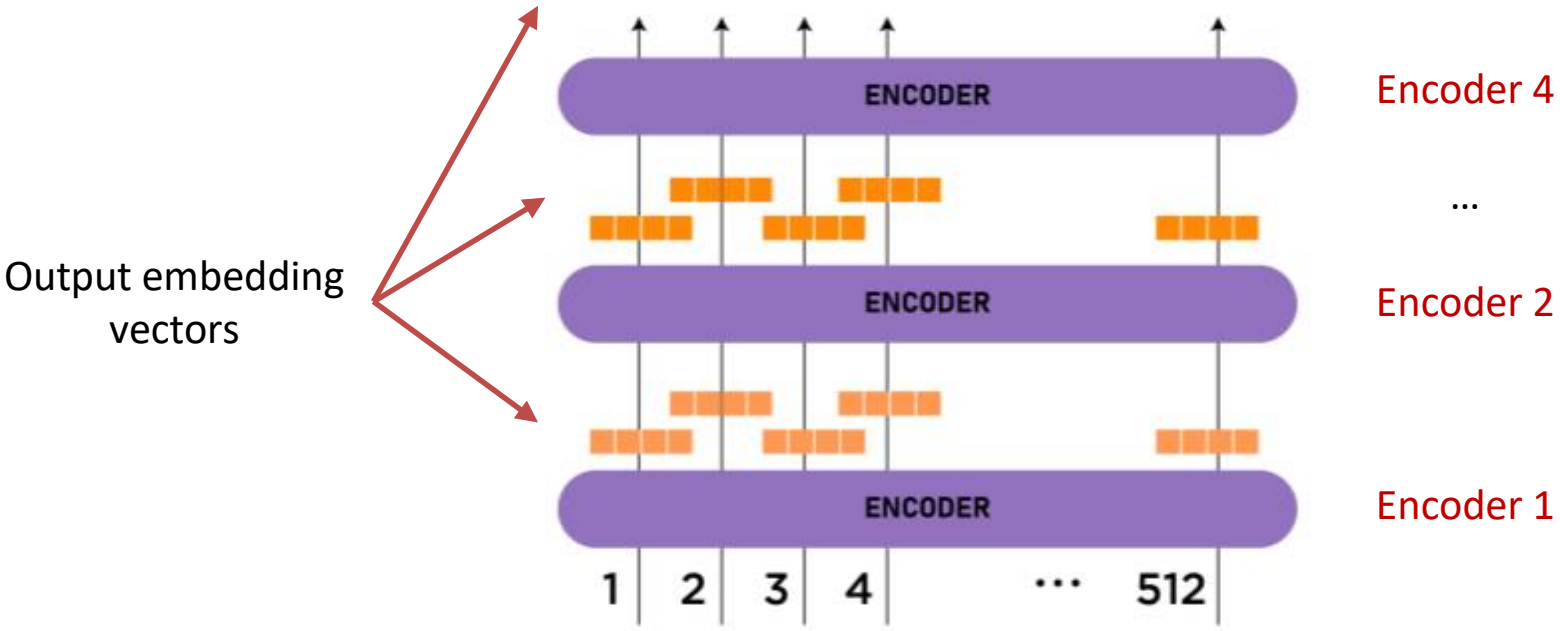
- Name: small_bert/bert_en_uncased_**L-4_H-128_A-2**
- Number of layers, i.e., Transformer-encoder blocks: 4.
- Number of hidden states: 128.
- Number of attention heads: 2.
- Model on TFHub: https://tfhub.dev/tensorflow/small_bert/bert_en_uncased_L-4_H-128_A-2/1
- Preprocessing module on TFHub: https://tfhub.dev/tensorflow/bert_en_uncased_preprocess/3



Assignment requirement

Since the BERT model has 4 encoder blocks and each outputs an embedding feature vector, your task is to find a way to make use of the **multi-layer features** to boost the classification accuracy. There are some questions you need to figure out:

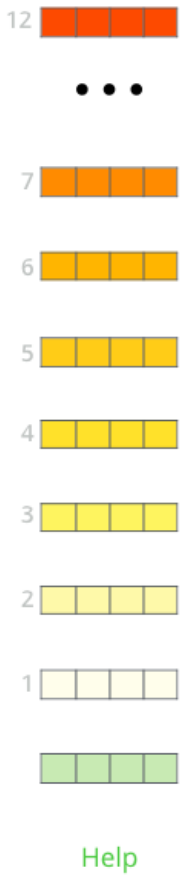
- How to **get embedding** feature vector of each encoder block.
- How to **combine the embedding** feature vectors of the encoder blocks.
- How to **make use of the combined embedding** feature vector to boost the model performance.

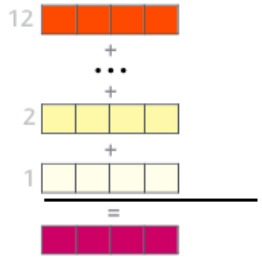
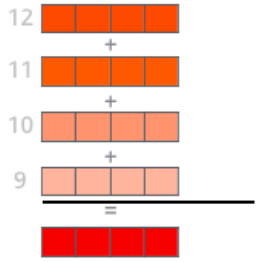
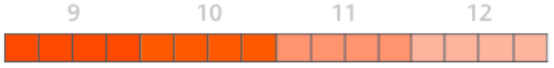


BERT with input sequence length of 512

BERT for feature extraction

What is the best contextualized embedding for "Help" in that context?
For named-entity recognition task CoNLL-2003 NER



		Dev F1 Score
First Layer		91.0
Last Hidden Layer		94.9
Sum All 12 Layers		95.5
Second-to-Last Hidden Layer		95.6
Sum Last Four Hidden		95.9
Concat Last Four Hidden		96.1

Assignment instruction

```
Model: "BERT_complete_model"
__________________________________________________________________________________________________
 Layer (type)                   Output Shape         Param #     Connected to
==================================================================================================
 text (InputLayer)              [(None,)]            0           []

 preprocessing (KerasLayer)     {'input_type_ids':   0           ['text[0][0]']
                                (None, 128),
                                 'input_mask': (Non
                                e, 128),
                                 'input_word_ids':
                                (None, 128)}

 BERT_encoder (KerasLayer)      {'encoder_outputs':  4782465     ['preprocessing[0][0]',
                                 [(None, 128, 128),               'preprocessing[0][1]',
                                 (None, 128, 128),                'preprocessing[0][2]']
                                 (None, 128, 128),
                                 (None, 128, 128)],
                                 'sequence_output':
                                 (None, 128, 128),
                                 'default': (None,
                                128),
                                 'pooled_output': (
                                None, 128)}

 dense (Dense)                  (None, 64)           8256        ['BERT_encoder[0][5]']

 batch_normalization (BatchNorm  (None, 64)          256         ['dense[0][0]']
 alization)

 dropout (Dropout)              (None, 64)           0           ['batch_normalization[0][0]']

 classifier (Dense)             (None, 1)            65          ['dropout[0][0]']

==================================================================================================
Total params: 4,791,042
Trainable params: 4,790,913
Non-trainable params: 129
```

small_bert/bert_en_uncased_L-4_H-128_A-2
Try to combine and use **encoder_output** embeddings as inputs of classification layers

Q&A

Thank you